

UNIVERSITY OF MANAGEMENT AND TECHNOLOGY

Open Source Software Development — OSSD Y9

PHASE 1 PROJECT DOCUMENTATION

Student Market Palace

A Student-Focused Buy & Sell Marketplace

Student Name	Arslan Saeed
Registration No	F2024408017
Course	Open Source Software Development
Semester	Fall 2026
Instructor	Sir Abdullah baigh
Submission Date	June2026

Phase 1 Deadline: 6th June 2026 | Phase 2: 19th June 2026 | Viva: 26th June 2026

1. Project Overview

1.1 Project Title

Student Market Palace

1.2 Problem Statement

University and college students regularly need to buy, sell, and exchange academic essentials — textbooks, handwritten notes, stationery, electronics, and personal accessories. General-purpose marketplaces such as OLX or Facebook Marketplace are not tailored to student contexts: they lack campus filtering, academic category taxonomies, and the peer-trust environment students require for safe, low-cost transactions.

Student Market Palace addresses this gap by providing a dedicated, student-verified platform where buyers and sellers share the same academic ecosystem, making transactions more transparent, affordable, and community-driven.

1.3 Project Objective

The primary objective of this project is to design, develop, and deploy a full-stack web application that:

- Enables students to register and authenticate securely.
- Allows sellers to list, edit, and remove academic and personal items.
- Empowers buyers to browse, search, filter, and inquire about listed products.
- Provides each user with a personalised dashboard for tracking activity.
- Stores all data reliably in a PostgreSQL database hosted on Supabase.
- Is deployed online with a live, accessible URL for evaluation.

1.4 Target Users

The platform is designed exclusively for:

- Undergraduate and postgraduate students who wish to sell unused or surplus academic items.
- Students seeking affordable second-hand books, notes, or electronics within their university community.
- University administrators or moderators who may oversee platform activity (future scope).

2. Features & Functional Requirements

2.1 Core Features

#	Feature	Description
1	User Registration & Login	Students register with name, email, and password. Passwords are hashed. JWT-based session management ensures secure access.
2	Product Listing Management	Authenticated users can create new listings with title, description, price, category, and images. Listings are persisted in the database.
3	Product Search & Filtering	Full-text search by keyword and multi-parameter filtering by category, price range, and availability status.
4	Product Details View	A dedicated page displaying complete product information, seller details, and a contact/inquiry option.
5	User Dashboard	Personalised overview showing active listings, sold items, purchase history, and marketplace statistics.
6	Profile Management	Users can update display name, contact information, and profile picture.
7	Product Update & Deletion	Sellers may edit listing details or remove listings they own. Ownership is enforced server-side.
8	Marketplace Statistics	A statistics endpoint returns aggregate data: total listings, categories, active users, and recent transactions.
9	Buy / Sell Marketplace	Buyers can mark interest and initiate a transaction, updating product status from 'available' to 'sold'.
10	Secure Data Management	All sensitive fields are encrypted/hashed; environment variables protect database credentials and JWT secrets.

2.2 Non-Functional Requirements

- Responsive design compatible with desktop, tablet, and mobile browsers.
- API response times under 500 ms for standard CRUD operations.
- Input validation and meaningful error messages on all API endpoints.
- Clean, accessible HTML/CSS frontend requiring no JavaScript framework.
- Code organised according to the recommended project folder structure.

3. Frontend Pages

The frontend consists of a minimum of seven HTML pages, each serving a distinct user function:

#	Page	Route / File	Purpose
1	Home Page	index.html	Landing page: hero banner, featured listings, call-to-action links, and platform statistics.
2	About Page	about.html	Project description, team details, and platform mission statement.
3	Login / Register	login.html	Tabbed form for new user registration and returning user login with validation feedback.
4	Dashboard Page	dashboard.html	Authenticated user's personal hub: active listings, purchase history, and quick statistics.
5	Manage Products	manage.html	CRUD interface: add a new listing, edit an existing one, or delete a product.
6	Product Details	product.html	Full details of a single listing with images, description, price, and seller contact option.
7	User Profile	profile.html	View and edit personal profile: name, email, contact info, and avatar.

4. Backend API Endpoints

The FastAPI backend exposes the following 12 REST endpoints. All endpoints return JSON. Protected routes require a valid JWT bearer token in the Authorization header.

#	Method	Endpoint	Auth	Description
1	POST	/register	Public	Register a new user account. Hashes password before storage.
2	POST	/login	Public	Authenticate user; returns JWT access token on success.
3	POST	/products	Protected	Create a new product listing linked to the authenticated user.
4	GET	/products	Public	Retrieve all available product listings with pagination support.
5	GET	/products/{id}	Public	Fetch complete details of a single product by its unique ID.
6	PUT	/products/{id}	Protected	Update listing details. Only the owning seller may edit.
7	DELETE	/products/{id}	Protected	Permanently remove a listing. Ownership verified before deletion.
8	GET	/products/search	Public	Full-text keyword search across product title and description fields.
9	GET	/products/filter	Public	Filter listings by category, price range, and/or availability status.
10	GET	/dashboard/stats	Protected	Return aggregate stats: total listings, sales, active users, category counts.
11	GET	/users/profile	Protected	Retrieve the authenticated user's profile information.

#	Method	Endpoint	Auth	Description
12	PUT	/users/profile	Protected	Update the authenticated user's name, email, or contact details.

Note: All endpoints include proper HTTP status codes (200, 201, 400, 401, 403, 404, 422, 500), input validation via Pydantic schemas, and structured JSON error responses.

5. Database Design

5.1 Database Choice

The project uses PostgreSQL hosted on Supabase. PostgreSQL was selected for its robust relational integrity, support for complex queries, and seamless integration with FastAPI via SQLAlchemy or the Supabase Python client. Supabase provides a free-tier cloud PostgreSQL instance with a built-in dashboard for schema management and monitoring.

5.2 Entity Relationship Summary

The schema comprises four core tables connected through foreign-key relationships:

- Users → Products (one-to-many: a user can create many listings)
- Products → Transactions (one-to-many: a product may appear in a transaction)
- Users → Transactions (one-to-many: a user may be the buyer of many transactions)
- Categories → Products (one-to-many: a category classifies many products)

5.3 Table Schemas

Table 1 — Users

Column	Data Type	Constraint	Description
user_id	SERIAL	PRIMARY KEY	Auto-incremented unique identifier for each user.
name	VARCHAR(100)	NOT NULL	Full display name of the registered student.
email	VARCHAR(150)	UNIQUE, NOT NULL	Unique email address used for login and contact.
password	TEXT	NOT NULL	Bcrypt-hashed password string — never stored as plain text.
role	VARCHAR(20)	DEFAULT 'student'	User role: 'student' (default) or 'admin'.
created_at	TIMESTAMP	DEFAULT NOW()	Timestamp of account creation.

Table 2 — Products

Column	Data Type	Constraint	Description
product_id	SERIAL	PRIMARY KEY	Auto-incremented unique identifier for each listing.
user_id	INTEGER	FOREIGN KEY (Users)	References the seller's user_id.
title	VARCHAR(200)	NOT NULL	Short, descriptive title of the listed item.
description	TEXT	NOT NULL	Detailed description of the product condition and specs.
price	DECIMAL(10,2)	NOT NULL	Asking price in PKR. Must be a positive value.
category	VARCHAR(100)	NOT NULL	Product category (e.g., Books, Electronics, Notes).
status	VARCHAR(20)	DEFAULT 'available'	Listing status: 'available', 'sold', or 'removed'.
created_at	TIMESTAMP	DEFAULT NOW()	Timestamp when the listing was first published.

Table 3 — Transactions

Column	Data Type	Constraint	Description
transaction_id	SERIAL	PRIMARY KEY	Auto-incremented unique transaction identifier.
buyer_id	INTEGER	FOREIGN KEY (Users)	References the purchasing student's user_id.
product_id	INTEGER	FOREIGN KEY (Products)	References the purchased product's product_id.
transaction_date	TIMESTAMP	DEFAULT NOW()	Date and time the transaction was recorded.

Table 4 — Categories

Column	Data Type	Constraint	Description
category_id	SERIAL	PRIMARY KEY	Auto-incremented unique identifier for each category.

Column	Data Type	Constraint	Description
category_name	VARCHAR(100)	UNIQUE, NOT NULL	Human-readable category label (e.g., Books, Laptops).

6. Technology Stack

Layer	Technology	Justification
Frontend	HTML5 & CSS3	Lightweight, dependency-free, and fully compatible with Vercel static hosting. JavaScript used minimally for form interactions.
Backend	FastAPI (Python)	High-performance async framework with automatic OpenAPI documentation (Swagger UI), Pydantic validation, and easy OAuth2/JWT integration.
Database	PostgreSQL	Robust relational RDBMS with strong ACID guarantees, well-suited for structured marketplace data with foreign-key relationships.
DB Hosting	Supabase	Managed cloud PostgreSQL with a free tier, web dashboard, and direct psycopg2-compatible connection strings.
ORM / DB Driver	SQLAlchemy / psycopg2	Pythonic database access layer enabling model-based queries and migration-free schema management during development.
Authentication	JWT (python-jose)	Stateless, scalable token-based authentication with configurable expiry and role-claim support.
Frontend Hosting	Vercel	Zero-config static deployment with global CDN and automatic HTTPS.
Backend Hosting	Render	Free-tier container hosting for Python web services with automatic GitHub deployments.
Version Control	Git & GitHub	Source code management and collaboration. Public repository required for submission.

7. Recommended Project Folder Structure

The following structure follows the course guidelines and separates concerns clearly between frontend, backend, and documentation assets:

StudentMarketPalace/

```

├── frontend/
│   ├── index.html           # Home page
│   ├── about.html          # About page
│   ├── login.html          # Login / Register
│   ├── dashboard.html      # User dashboard
│   ├── manage.html         # Manage products (CRUD)
│   ├── product.html        # Product details view
│   ├── profile.html        # User profile
│   └── style.css            # Shared stylesheet
├── backend/
│   ├── main.py              # FastAPI app entry point & route registration
│   ├── models.py            # SQLAlchemy ORM models
│   ├── schemas.py           # Pydantic request/response schemas
│   ├── database.py          # DB connection & session factory
│   ├── auth.py              # JWT creation & verification utilities
│   └── routers/
│       ├── users.py         # User registration & profile routes
│       ├── products.py      # Product CRUD routes
│       └── dashboard.py     # Statistics route
│   ├── .env                 # Environment variables (not committed to Git)
│   └── requirements.txt     # Python dependencies
├── screenshots/             # Deployment & DB screenshots for submission
├── README.md                # Setup instructions
└── .gitignore               # Excludes .env, __pycache__, etc.
```

8. Deployment Plan

All three application layers will be hosted on free-tier cloud platforms. The deployment procedure is outlined below:

Layer	Platform	Configuration Notes
Frontend	Vercel	Connect GitHub repository; set root directory to /frontend. Auto-deploy on push to main branch.
Backend	Render	Create a new Web Service; set build command to <code>pip install -r requirements.txt</code> and start command to <code>uvicorn main:app --host 0.0.0.0 --port 10000</code> .
Database	Supabase	Create a new Supabase project; copy the connection string into the backend .env file as <code>DATABASE_URL</code> .

8.1 Step-by-Step Deployment Checklist

1. Push complete project to a public GitHub repository.
2. Create a Supabase project and define all four tables via the SQL Editor or Table Editor.
3. Add `DATABASE_URL`, `SECRET_KEY`, and `ALGORITHM` as Render environment variables.
4. Deploy the FastAPI backend on Render; verify all API endpoints via the auto-generated /docs (Swagger UI).
5. Deploy the frontend on Vercel; update all API base URLs in HTML/JS files to point to the Render backend URL.
6. Run end-to-end tests: register a user, create a listing, search, update, and delete.
7. Capture deployment screenshots and include them in the /screenshots folder.
8. Update README.md with the live frontend URL, backend API URL, and setup instructions.

8.2 Submission Deliverables

- Phase 1 documentation PDF (this document)
- Complete project source code (GitHub repository link)
- Live frontend deployment link (Vercel URL)
- Live backend API link (Render URL + /docs Swagger UI)
- Database screenshots (Supabase table views and configuration)
- Project screenshots (at least one per page)
- README.md with local setup instructions

9. Expected Output

Upon successful completion of all three project phases, the following outcomes are expected:

- A fully functional, publicly accessible Student Market Palace web application deployable from any browser without local setup.
- A FastAPI backend serving all 12 REST API endpoints, documented automatically at /docs (Swagger UI) and /redoc.
- A PostgreSQL database on Supabase storing real user, product, and transaction records with referential integrity enforced.
- A clean, responsive multi-page HTML/CSS frontend requiring no JavaScript framework installation.
- Full CRUD operations demonstrable through both the UI and API directly.
- A GitHub repository with organised code, a descriptive README, and no sensitive credentials committed.

Minimum Checklist Confirmation:

Requirement	Status
Frontend has at least 5 pages (7 planned)	✓ Met
Backend has at least 10 API endpoints (12 planned)	✓ Met
At least 1 database (PostgreSQL on Supabase)	✓ Met
Proper CRUD functionality across all major entities	✓ Met
Clear database schema with 4 normalised tables	✓ Met
Deployment plan defined for frontend, backend & DB	✓ Met
Documentation covers all required Phase 1 sections	✓ Met